CIS 3300 · THE SPEEDRUN LEADERBOARD

Checkpoint 4

Grouping Runs by Board

Starts from: SpeedrunCheckpoint3. **Ends at:** SpeedrunCheckpoint4. **Prepares you for:** Lab W14, Part 2: choosing the collections.

At the end of Checkpoint 3, the program showed one leaderboard, and the game and category were typed into the code. A real leaderboard site has a board for every game and category, and it knows who its players are. In this checkpoint, the program groups the runs by board, keeps the set of players, and builds a leaderboard for every board. Each of those jobs needs a different kind of collection, and choosing the right one is the point of this checkpoint.

Before you start, see the *Getting Started* guide for how to import a project into Eclipse and for the rules the leaderboard uses for each field.

A note about this guide. I developed this guide with help from AI, which drafted much of the code, data, and explanations, and I review and revise them. Every program was compiled and run, and every output shown is real. If something looks wrong, does not match what you see in Eclipse, or contradicts the book or the notes, tell me. This guide shows how to build one program. It does not replace the notes, the assigned chapters, or the recorded lectures.

Step 1: Background: choosing a collection

Java's collections come in three main kinds, each an interface with several classes that implement it. Table 1 lists the ones this checkpoint uses.

Table 1: The collections used in this checkpoint.

Interface	Class	What it does
`List`	`ArrayList`	Keeps elements in the order you add them, duplicates allowed, and finds them by position.
`Set`	`HashSet`	Keeps one copy of each element, in no useful order.
`Set`	`TreeSet`	Keeps one copy of each element, in natural order.
`Map`	`HashMap`	Finds a value by its key, with keys in no useful order.
`Map`	`TreeMap`	Finds a value by its key, with keys in natural order.

To choose one, ask what the code needs to do with it. Look things up by a key? A `Map`. Keep only one of each? A `Set`. Keep everything in an order? A `List`. Then choose the class: a *Tree* class keeps its elements or keys sorted, using `compareTo`, and a *Hash* class does not.

Declare each variable with the interface type and create it with the class, using the diamond `<>` so the compiler fills in the type arguments:

```
Map<Board, List<Run>> runsByBoard = new TreeMap<>();
```

The rest of the program then works with any `Map`, and changing to a different class means changing one word.

Step 2: A key for each board

The runs will be grouped in a map whose key is the board, a game and a category together. In the `speedrun` package, create a new record named `Board`:

```
package speedrun;

public record Board(String game, String category) implements Comparable<Board> {

    @Override
    public int compareTo(Board other) {
        int byGame = game.compareTo(other.game);
        if (byGame != 0) {
            return byGame;
        }
        return category.compareTo(other.category);
    }

    @Override
    public String toString() {
        return game + ", " + category;
    }
}
```

- **A map must be able to tell when two keys are equal.** A `HashMap` uses `equals` and `hashCode` to find a key, and a record writes both for you, comparing every component. Two `Board` objects for Celeste, Any% are equal, so they find the same entry.
- **A `TreeMap` key also needs `compareTo`,** which is how it keeps its keys in order. Boards sort by game, then by category, the same pattern as `Run`'s `compareTo` in Checkpoint 3.
- **A record may replace a method Java wrote for it.** Here `toString` returns `Celeste, Any\%` instead of `Board[game=Celeste, category=Any\%]`.

Then give `Run` a method that returns its board. In `Run.java`, add this above `compareTo`:

```
public Board board() {
    return new Board(game, category);
}
```

Step 3: Group the runs by board

Add this method to `SpeedrunReport`, below `main`, with imports for `java.util.Map`, `java.util.TreeMap`, and `speedrun.Board`:

```
private static Map<Board, List<Run>> groupByBoard(List<Run> runs) {
    Map<Board, List<Run>> result = new TreeMap<>();
    for (Run run : runs) {
        List<Run> list = result.get(run.board());
```

```
        if (list == null) {
            list = new ArrayList<>();
            result.put(run.board(), list);
        }
        list.add(run);
    }
    return result;
}
```

For each run, `get` looks for the list of runs on that run's board. The first time a board appears, there is no list yet, so `get` returns `null`, and the method creates an empty list and `put`s it in the map. Either way, the run is then added to its board's list. The value type is `List<Run>` because each board holds many runs, in the order they were submitted.

In `main`, delete the Celeste code from Checkpoint 3 (the `Leaderboard<Run> celeste` lines and the call to `printBoard`), and put this in its place:

```
Map<Board, List<Run>> runsByBoard = groupByBoard(runs);
for (Map.Entry<Board, List<Run>> entry : runsByBoard.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue().size() + " runs");
}
```

A map is not a list, so it cannot be looped over directly. `entrySet()` returns its entries, and each `Map.Entry` holds one key and its value, which `getKey()` and `getValue()` return. Run the program:

```
Celeste, Any%: 8 runs
Hollow Knight, Any%: 7 runs
Portal, Inbounds: 7 runs
Super Mario 64, 120 Star: 7 runs
Super Mario 64, 16 Star: 6 runs
The Legend of Zelda: Ocarina of Time, 100%: 6 runs
```

The boards are in the order `Board`'s `compareTo` gives them: by game, then by category. The counts add up to 41, every good run. Notice that `120 Star` comes before `16 Star`: strings compare one character at a time, and the second characters, `2` and `6`, decide, not the numbers 120 and 16.

Step 4: Why a TreeMap?

To see what the `TreeMap` is doing, change it to a `HashMap` in `groupByBoard`, one word plus an import, and run the program again:

```
Hollow Knight, Any%: 7 runs
Celeste, Any%: 8 runs
Portal, Inbounds: 7 runs
The Legend of Zelda: Ocarina of Time, 100%: 6 runs
Super Mario 64, 16 Star: 6 runs
Super Mario 64, 120 Star: 7 runs
```

The counts are the same, because a `HashMap` finds keys just as well. But the order is not useful, and Java does not promise it will even stay the same. A report should list its boards in a predictable order, so change it back to a `TreeMap`.

A `TreeMap` depends on its keys' `compareTo`, and the compiler does not check that the key type has one. If `Board` did not implement `Comparable`, the program would still compile, and then fail the first time the map compared a key. The message is shortened here:

```
Exception in thread "main" java.lang.ClassCastException: class speedrun.Board
cannot be cast to class java.lang.Comparable (...)
```

Step 5: The players, as a set

Each player can appear on many runs, but the list of players should name each player once. That is exactly what a `Set` does. In `main`, right before the line that calls `groupByBoard`, add these lines, with imports for `java.util.Set` and `java.util.HashSet`:

```
Set<String> players = new HashSet<>();
for (Run run : runs) {
    players.add(run.player());
}
System.out.println(players.size() + " players:");
System.out.println(players);
```

```
10 players:
[NovaByte, Moss, GrapeSoda, Lumen, Kestrel, PixelPilot, Tangent, Vela, Sprocket, Rook]
```

`add` was called 41 times, once for every run, but the set holds 10 names. When a name is already in the set, `add` leaves the set unchanged and returns `false`. The order, as with the `HashMap`, is not useful. Change `HashSet` to `TreeSet`, and the import with it:

```
10 players:
[GrapeSoda, Kestrel, Lumen, Moss, NovaByte, PixelPilot, Rook, Sprocket, Tangent, Vela]
```

Now the players are in alphabetical order, the natural order of `String`.

Step 6: A leaderboard for every board

Now replace the loop from Step 3, the one that prints the counts, with a loop that builds a `Leaderboard<Run>` for each board:

```
Map<Board, List<Run>> runsByBoard = groupByBoard(runs);
for (Map.Entry<Board, List<Run>> entry : runsByBoard.entrySet()) {
    Leaderboard<Run> top = new Leaderboard<>(3);
    for (Run run : entry.getValue()) {
        top.add(run);
    }
    System.out.println();
    printBoard(entry.getKey() + ": " + entry.getValue().size() + " runs, top 3", top);
}
```

Change `printBoard` so that it prints the title as given, and marks runs that are not verified:

```
private static void printBoard(String title, Leaderboard<Run> board) {
    System.out.println(title);
    List<Run> entries = board.getEntries();
    for (int i = 0; i < entries.size(); i++) {
        Run run = entries.get(i);
        System.out.printf("%2d. %-10s %10s  %s%s%n", i + 1, run.player(),
                RunTime.format(run.time()), run.date(),
                run.verified() ? "" : "  (unverified)");
    }
}
```

Run the program. The output begins like this, and continues through all six boards:

```
10 players:
[GrapeSoda, Kestrel, Lumen, Moss, NovaByte, PixelPilot, Rook, Sprocket, Tangent, Vela]

Celeste, Any%: 8 runs, top 3
 1. Kestrel       26:41.6  2026-02-14
 2. Moss          26:41.6  2026-05-03
 3. Vela          27:17.7  2026-05-20

Hollow Knight, Any%: 7 runs, top 3
 1. NovaByte      33:34.8  2026-01-31
 2. NovaByte      34:36.7  2026-01-21
 3. Moss          38:16.5  2026-04-29

Portal, Inbounds: 7 runs, top 3
 1. Tangent        9:12.8  2026-07-21  (unverified)
 2. Kestrel       10:20.1  2026-08-14
 3. PixelPilot    10:30.1  2026-02-05
```

One class from Checkpoint 3 now serves six boards. Two things in this output are not right yet, and the next checkpoints fix them:

- **Unverified runs lead three of the six boards,** including Portal. A leaderboard should count only verified runs. Checkpoint 5 fixes that.
- **NovaByte holds two of the three Hollow Knight places.** A player's slower runs should not push other players off the board; a board should show each player's *personal best*. Checkpoint 6 fixes that.

Step 7: No raw types

Every collection in this program says what it holds: `List<Run>`, `Set<String>`, `Map<Board, List<Run>>`. Leaving the type arguments off creates a *raw type*, which is how collections were written before Java had generics. A raw collection holds plain `Object`s, so the compiler cannot check what goes in or what comes out:

```
List rawRuns = new ArrayList();
rawRuns.add(run);
Run first = rawRuns.get(0);
```

```
warning: [unchecked] unchecked call to add(E) as a member of the raw type List
error: incompatible types: Object cannot be converted to Run
```

The `add` is allowed, with only a warning, so anything at all could be put in the list. Taking the run back out fails, because the compiler only knows it is an `Object`. Eclipse marks every raw type with a warning. Treat that warning as a mistake to fix: always write the type arguments, or the diamond `<>` after `new`.

Your project should now match SpeedrunCheckpoint4.

Check yourself

1. Why is `runsByBoard` a `Map` rather than a `List`? Why are its values lists?
2. Why does `Super Mario 64, 120 Star` come before `Super Mario 64, 16 Star`?
3. Without `compareTo`, `Board` still compiles as a `TreeMap` key, and the program fails only when it runs. Why does the compiler not catch it?
4. `players.add` is called 41 times, but the set ends up with 10 names. What happens when a name is added a second time?
5. What is wrong with declaring a list as `List rawRuns = new ArrayList()`, and how do you fix it?

Answers

1. The program needs to find the runs for a given board, which is a lookup by key. Each board has many runs, kept in the order they were submitted, so each value is a `List<Run>`.
2. Strings compare one character at a time. The first difference is the second character, `2` in `120` and `6` in `16`, and `2` comes first. Strings are not compared as numbers.
3. `TreeMap` puts no bound on its key type, so the compiler accepts any key. When the map needs to compare two keys, it treats the key as a `Comparable`, and that fails at run time for a class that does not implement it. `Leaderboard`, which does bound its type, catches the same mistake at compile time.
4. Nothing changes. `add` sees the name is already in the set, leaves the set as it is, and returns `false`.
5. It is a raw type, so the list holds plain `Objects`: anything can go in, and every element must be cast on the way out. Write `List<Run> runs = new ArrayList<>();` instead.